UTS

PEMBELAJARAN MESIN



NAMA : Sony Abdullah Maulidan Tuasikal

NIM : 202431054

KELAS : F

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 03

ASISTEN : 1. Jonathan Juan Setyanto

2. Muhammad Caisar Gemilang Pratama

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

HALAMAN JUDUL i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Rumusan Masalah 1

1.2 Tujuan Masalah 2

1.3 Manfaat Masalah 2

BAB II LANDASAN TEORI 3

2.1 Pembelajaran Mesin (Machine Learning) 3

2.2 Supervised Learning 4

2.3 Klasifikasi (Classification) 4

2.4 Dataset Heart Disease 5

2.5 Exploratory Data Analysis (EDA) 6

2.6 Data Preprocessing 7

2.6.1 Penanganan Missing Value 7

2.6.2 Pemisahan Fitur dan Target 8

2.6.3 Encoding Data Kategorikal 8

2.6.4 Pembagian Data Latih dan Data Uji 9

2.7 Algoritma Naive Bayes 9

2.8 Algoritma Decision Tree 10

2.9 Evaluasi Model Klasifikasi 11

2.9.1 Confusion Matrix 11

2.9.2 Accuracy 12

2.9.3 Precision 12

2.9.4 Recall 13

2.9.5 F1-Score 13

2.10 Penelitian Terdahulu 14

BAB III HASIL 15

3.1 Lingkungan dan Persiapan Praktikum 15

3.2 Import Library dan Membaca Dataset 15

3.3 Exploratory Data Analysis (EDA) 16

3.3.1 Melihat Sampel Dataset 16

3.3.2 Memeriksa Ukuran dan Nama Kolom 16

3.3.3 Memeriksa Tipe Data 17

3.3.4 Memeriksa Missing Value dan Data Duplikat 17

3.3.5 Statistik Deskriptif 18

3.3.6 Visualisasi Distribusi Data 18

3.3.7 Distribusi Target HeartDisease 19

3.3.8 Pemeriksaan Nilai Tidak Valid dan Outlier 19

3.4 Data Preprocessing 20

3.4.1 Pemisahan Fitur dan Target 20

3.4.2 Pembagian Data Latih dan Data Uji 21

3.4.3 Penanganan Data dan Encoding 21

3.5 Implementasi Algoritma Naive Bayes 22

3.6 Implementasi Algoritma Decision Tree 24

3.7 Perbandingan Hasil Evaluasi Model 26

3.8 Analisis Hasil Praktikum 27

BAB IV PENUTUP 28

4.1 Kesimpulan 28

DAFTAR PUSTAKA 29

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Penyakit jantung merupakan salah satu masalah kesehatan yang memerlukan perhatian karena dapat berdampak serius terhadap kondisi kesehatan seseorang. Perkembangan teknologi pembelajaran mesin memungkinkan data kesehatan dianalisis untuk membantu melakukan klasifikasi atau prediksi berdasarkan pola yang terdapat di dalam data.

Pada praktikum Ujian Tengah Semester (UTS) Pembelajaran Mesin ini, digunakan dataset Heart Disease yang berisi 918 data dengan 12 kolom, termasuk atribut karakteristik pasien dan kolom target HeartDisease. Penelitian ini menerapkan dua algoritma klasifikasi, yaitu Naive Bayes dan Decision Tree.

Berdasarkan latar belakang tersebut, rumusan masalah dalam praktikum ini adalah:

1. Bagaimana melakukan eksplorasi dan preprocessing terhadap dataset Heart Disease?
2. Bagaimana menerapkan algoritma Naive Bayes untuk mengklasifikasikan data penyakit jantung?
3. Bagaimana menerapkan algoritma Decision Tree untuk mengklasifikasikan data penyakit jantung?
4. Bagaimana mengukur performa kedua algoritma menggunakan accuracy, precision, recall, dan F1-score?
5. Algoritma manakah yang menghasilkan akurasi lebih tinggi berdasarkan hasil pengujian?

1.2 Tujuan Masalah

Tujuan dari pelaksanaan praktikum UTS Pembelajaran Mesin ini adalah:

1. Memahami karakteristik dataset Heart Disease melalui proses Exploratory Data Analysis (EDA).
2. Melakukan preprocessing data agar dapat digunakan dalam proses pemodelan.
3. Mengimplementasikan algoritma Naive Bayes dan Decision Tree menggunakan bahasa pemrograman Python.
4. Melakukan pelatihan dan pengujian model menggunakan pembagian data latih dan data uji.
5. Mengevaluasi performa model menggunakan accuracy, precision, recall, dan F1-score.
6. Membandingkan hasil klasifikasi kedua algoritma untuk mengetahui algoritma dengan performa terbaik pada eksperimen ini.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah:

1. Menambah pemahaman mengenai konsep dasar pembelajaran mesin, khususnya supervised learning untuk klasifikasi.

2. Memberikan pengalaman dalam melakukan eksplorasi, pengolahan, dan preprocessing dataset menggunakan Python.
3. Memahami penerapan algoritma Naive Bayes dan Decision Tree dalam proses klasifikasi data.
4. Mengetahui cara mengevaluasi performa model menggunakan beberapa metrik evaluasi.
5. Menambah wawasan mengenai pemanfaatan pembelajaran mesin dalam analisis data kesehatan, khususnya klasifikasi penyakit jantung.

BAB II

LANDASAN TEORI

2.1 Pembelajaran Mesin (Machine Learning)

Pembelajaran mesin atau *Machine Learning* merupakan salah satu cabang dari kecerdasan buatan (*Artificial Intelligence*) yang memungkinkan komputer mempelajari pola dari data untuk menghasilkan prediksi atau keputusan tanpa harus diprogram secara eksplisit untuk setiap kondisi. Machine Learning bekerja dengan memanfaatkan data sebagai bahan pembelajaran sehingga sistem dapat mengenali hubungan antarvariabel dan menghasilkan model yang dapat digunakan untuk menganalisis data baru.

Dalam bidang kesehatan, Machine Learning dapat dimanfaatkan untuk menganalisis data pasien, mengidentifikasi pola penyakit, serta membantu proses prediksi berdasarkan informasi klinis. Menurut Mao et al. (2025), berbagai algoritma Machine Learning telah diterapkan dalam penelitian prediksi dan diagnosis penyakit jantung dengan memanfaatkan data klinis untuk mengenali pola yang berkaitan dengan kondisi penyakit.

Pada praktikum ini, Machine Learning diterapkan untuk melakukan klasifikasi penyakit jantung menggunakan dataset Heart Disease. Algoritma yang digunakan adalah Naive Bayes dan Decision Tree.

2.2 Supervised Learning

Supervised Learning merupakan metode pembelajaran mesin yang menggunakan data berlabel sebagai bahan pelatihan. Data tersebut terdiri atas variabel input atau fitur dan variabel target yang menjadi hasil atau kelas yang ingin diprediksi.

Pada proses pembelajaran, algoritma mempelajari hubungan antara fitur dan label dari data latih. Setelah proses pelatihan selesai, model digunakan untuk memprediksi label pada data baru yang belum pernah dipelajari sebelumnya.

Supervised Learning umumnya digunakan untuk dua jenis permasalahan, yaitu klasifikasi dan regresi. Klasifikasi digunakan untuk memprediksi kategori atau kelas, sedangkan regresi digunakan untuk memprediksi nilai numerik.

Dalam praktikum ini, Supervised Learning digunakan karena dataset Heart Disease memiliki kolom target bernama HeartDisease yang menunjukkan kategori kondisi penyakit jantung. Nilai 0 menunjukkan tidak terdapat penyakit jantung berdasarkan label dataset, sedangkan nilai 1 menunjukkan terdapat penyakit jantung berdasarkan label dataset.

2.3 Klasifikasi (Classification)

Klasifikasi merupakan salah satu teknik dalam Supervised Learning yang bertujuan mengelompokkan data ke dalam kelas tertentu berdasarkan pola dan karakteristik fitur yang dimiliki. Proses klasifikasi dilakukan dengan melatih model menggunakan data yang telah memiliki label, kemudian model tersebut digunakan untuk menentukan kelas dari data yang belum diketahui labelnya.

Pada penelitian terkait penyakit jantung, klasifikasi dapat digunakan untuk membedakan data pasien ke dalam kategori tertentu berdasarkan atribut seperti usia, jenis kelamin, tekanan darah, kolesterol, dan karakteristik medis lainnya.

Duhok dan Abdulazeez (2024) menjelaskan bahwa berbagai algoritma klasifikasi Machine Learning, termasuk Naive Bayes dan Decision Tree, telah banyak dikaji untuk permasalahan klasifikasi penyakit jantung.

Dalam praktikum ini, klasifikasi dilakukan dengan menggunakan dua kelas target, yaitu:

1. Kelas 0: Tidak terdapat penyakit jantung berdasarkan label dataset.
2. Kelas 1: Terdapat penyakit jantung berdasarkan label dataset.

2.4 Dataset Heart Disease

Dataset merupakan kumpulan data yang digunakan sebagai bahan analisis dan proses pembelajaran mesin. Dataset Heart Disease merupakan kumpulan data yang berisi atribut atau karakteristik pasien yang berkaitan dengan kondisi kesehatan jantung.

Dataset yang digunakan dalam praktikum ini memiliki 918 baris data dan 12 kolom. Sebanyak 11 kolom digunakan sebagai fitur atau variabel input, sedangkan satu kolom digunakan sebagai target klasifikasi.

Atribut yang terdapat dalam dataset antara lain:

1. **Age**: Usia pasien.
2. **Sex**: Jenis kelamin pasien.
3. **ChestPainType**: Jenis nyeri dada.
4. **RestingBP**: Tekanan darah saat istirahat.
5. **Cholesterol**: Kadar kolesterol.
6. **FastingBS**: Kondisi gula darah puasa berdasarkan nilai pada dataset.
7. **RestingECG**: Hasil elektrokardiogram saat istirahat.
8. **MaxHR**: Denyut jantung maksimum yang dicapai.
9. **ExerciseAngina**: Keterangan angina saat berolahraga.
10. **Oldpeak**: Nilai depresi segmen ST akibat aktivitas fisik dibandingkan kondisi istirahat.
11. **ST_Slope**: Kemiringan segmen ST saat aktivitas puncak.
12. **HeartDisease**: Label target yang menunjukkan kategori penyakit jantung.

Dataset tersebut digunakan untuk mempelajari hubungan antara karakteristik pasien dan label HeartDisease sehingga model dapat melakukan klasifikasi terhadap data pengujian.

2.5 Exploratory Data Analysis (EDA)

Exploratory Data Analysis atau EDA merupakan proses eksplorasi data yang bertujuan memahami karakteristik, pola, distribusi, dan kondisi data sebelum dilakukan pemodelan. EDA membantu mengetahui kualitas dataset serta menemukan permasalahan yang dapat memengaruhi hasil analisis.

Beberapa tahapan yang umum dilakukan dalam EDA meliputi:

1. **Pemeriksaan ukuran data**, untuk mengetahui jumlah baris dan kolom pada dataset.
2. **Pemeriksaan tipe data**, untuk mengetahui jenis data pada setiap atribut.
3. **Pemeriksaan missing value**, untuk mengetahui apakah terdapat nilai yang kosong.
4. **Pemeriksaan data duplikat**, untuk mengetahui apakah terdapat baris data yang berulang.
5. **Analisis statistik deskriptif**, untuk mengetahui nilai rata-rata, standar deviasi, nilai minimum, dan maksimum.
6. **Visualisasi data**, untuk melihat distribusi dan hubungan antarvariabel.

Dalam praktikum ini, EDA digunakan untuk memahami karakteristik dataset Heart Disease sebelum dilakukan preprocessing dan pelatihan model.

2.6 Data Preprocessing

Data Preprocessing merupakan tahapan pengolahan data sebelum digunakan dalam proses pelatihan Machine Learning. Tahapan ini bertujuan mempersiapkan data agar sesuai dengan kebutuhan algoritma serta mengurangi permasalahan yang dapat mengganggu proses pembelajaran.

Beberapa proses preprocessing yang digunakan atau relevan dalam praktikum ini adalah sebagai berikut.

2.6.1 Penanganan Missing Value

Missing value merupakan kondisi ketika suatu atribut tidak memiliki nilai atau berisi data kosong. Keberadaan missing value dapat memengaruhi proses pelatihan model sehingga perlu ditangani sebelum pemodelan.

Salah satu metode penanganannya adalah imputasi, yaitu mengganti nilai yang kosong menggunakan nilai tertentu. Pada fitur numerik, imputasi dapat dilakukan menggunakan median, sedangkan pada fitur kategorikal dapat menggunakan modus atau nilai yang paling sering muncul.

2.6.2 Pemisahan Fitur dan Target

Pemisahan fitur dan target merupakan proses membedakan variabel input dengan variabel yang akan diprediksi.

Pada dataset Heart Disease, kolom HeartDisease digunakan sebagai target, sedangkan 11 kolom lainnya digunakan sebagai fitur. Pemisahan ini diperlukan agar model mempelajari hubungan antara karakteristik data dengan kelas target.

2.6.3 Encoding Data Kategorikal

Encoding merupakan proses mengubah data kategorikal menjadi representasi numerik agar dapat diproses oleh algoritma Machine Learning.

Salah satu teknik encoding adalah *One-Hot Encoding*. Teknik ini mengubah setiap kategori menjadi kolom indikator yang berisi nilai 0 atau 1. Dengan demikian, data kategorikal dapat digunakan dalam proses pelatihan model tanpa memberikan urutan angka yang tidak sesuai dengan makna kategorinya.

2.6.4 Pembagian Data Latih dan Data Uji

Pembagian data atau *train-test split* merupakan proses membagi dataset menjadi data latih (*training data*) dan data uji (*testing data*).

Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk mengevaluasi kemampuan model terhadap data yang tidak digunakan selama proses pelatihan.

Pada praktikum ini, dataset dibagi dengan perbandingan 80% data latih dan 20% data uji. Pembagian dilakukan menggunakan teknik stratifikasi agar proporsi kelas target tetap terjaga pada kedua bagian data.

2.7 Algoritma Naive Bayes

Naive Bayes merupakan algoritma klasifikasi probabilistik yang didasarkan pada Teorema Bayes. Algoritma ini menghitung probabilitas suatu kelas berdasarkan bukti atau fitur yang terdapat pada data.

Naive Bayes memiliki asumsi bahwa setiap fitur bersifat independen terhadap fitur lainnya apabila kelasnya telah diketahui. Asumsi tersebut membuat algoritma ini relatif sederhana dan efisien untuk digunakan dalam permasalahan klasifikasi.

Dalam penerapannya, Naive Bayes menghitung probabilitas setiap kelas berdasarkan fitur yang dimiliki oleh suatu data, kemudian menentukan kelas dengan probabilitas paling tinggi.

Pada praktikum ini digunakan Gaussian Naive Bayes (GaussianNB). Algoritma tersebut merupakan varian Naive Bayes yang digunakan untuk fitur numerik dengan asumsi bahwa nilai fitur pada setiap kelas mengikuti distribusi Gaussian atau distribusi normal.

Penggunaan Gaussian Naive Bayes pada dataset Heart Disease bertujuan mengklasifikasikan data berdasarkan karakteristik pasien untuk menentukan kategori target HeartDisease.

2.8 Algoritma Decision Tree

Decision Tree merupakan algoritma Machine Learning yang menggunakan struktur berbentuk pohon untuk melakukan pengambilan keputusan dan klasifikasi. Struktur tersebut terdiri atas *root node*, *internal node*, cabang, dan *leaf node*.

Root node merupakan titik awal pembagian data, internal node merupakan titik pengujian berdasarkan atribut tertentu, cabang menunjukkan hasil dari pengujian, sedangkan leaf node merupakan hasil akhir berupa kelas prediksi.

Decision Tree membentuk pohon dengan memilih atribut yang dianggap paling baik untuk memisahkan data ke dalam kelompok kelas. Salah satu metode pengukuran yang dapat digunakan adalah Gini Impurity.

Gini Impurity digunakan untuk mengukur tingkat ketidakmurnian suatu kelompok data. Semakin kecil nilai Gini Impurity, semakin tinggi tingkat kemurnian kelas dalam kelompok tersebut.

Pada praktikum ini digunakan algoritma Decision Tree Classifier dengan kriteria Gini. Model dilatih menggunakan fitur-fitur pada dataset Heart Disease untuk menghasilkan aturan keputusan yang dapat digunakan dalam proses klasifikasi.

2.9 Evaluasi Model Klasifikasi

Evaluasi model merupakan proses untuk mengukur kemampuan model dalam melakukan prediksi terhadap data uji. Evaluasi diperlukan agar performa algoritma dapat diketahui dan dibandingkan secara objektif.

Dalam praktikum ini, evaluasi dilakukan menggunakan beberapa metrik, yaitu accuracy, precision, recall, F1-score, dan confusion matrix.

2.9.1 Confusion Matrix

Confusion Matrix merupakan tabel yang digunakan untuk membandingkan hasil prediksi model dengan kelas sebenarnya. Pada klasifikasi dua kelas, confusion matrix terdiri atas empat komponen:

1. **True Positive (TP)**: Data positif yang berhasil diprediksi sebagai positif.
2. **True Negative (TN)**: Data negatif yang berhasil diprediksi sebagai negatif.
3. **False Positive (FP)**: Data negatif yang salah diprediksi sebagai positif.
4. **False Negative (FN)**: Data positif yang salah diprediksi sebagai negatif.

Confusion matrix membantu mengetahui jenis kesalahan yang dilakukan model dalam proses klasifikasi.

2.9.2 Accuracy

Accuracy merupakan metrik yang mengukur persentase jumlah prediksi yang benar dibandingkan dengan seluruh jumlah data yang diuji.

Accuracy dihitung dengan rumus:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \times 100\%$$

Semakin tinggi nilai accuracy, semakin besar proporsi prediksi yang benar terhadap seluruh data pengujian.

2.9.3 Precision

Precision merupakan metrik yang mengukur ketepatan model ketika memprediksi suatu data sebagai kelas positif.

Precision dihitung dengan rumus:

$$\text{Precision} = TP / (TP + FP)$$

Nilai precision yang tinggi menunjukkan bahwa sebagian besar prediksi positif yang dihasilkan model memang benar termasuk kelas positif.

2.9.4 Recall

Recall merupakan metrik yang mengukur kemampuan model dalam menemukan seluruh data yang benar-benar termasuk kelas positif.

Recall dihitung dengan rumus:

Recall = TP / (TP + FN)

Nilai recall yang tinggi menunjukkan bahwa model mampu mengenali sebagian besar data positif yang sebenarnya.

2.9.5 F1-Score

F1-score merupakan metrik yang menggabungkan precision dan recall menggunakan rata-rata harmonik. Metrik ini berguna untuk menilai keseimbangan antara ketepatan prediksi positif dan kemampuan menemukan data positif.

F1-score dihitung dengan rumus:

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

Semakin tinggi nilai F1-score, semakin baik keseimbangan antara precision dan recall yang dihasilkan model.

2.10 Penelitian Terdahulu

Penelitian terdahulu digunakan sebagai dasar pendukung dalam penerapan Machine Learning untuk klasifikasi dan prediksi penyakit jantung.

Mao et al. (2025) melakukan kajian sistematis mengenai algoritma Machine Learning untuk diagnosis penyakit jantung. Penelitian tersebut membahas penggunaan beberapa algoritma supervised learning, termasuk Decision Tree dan Naive Bayes, dalam analisis data penyakit jantung.

Duhok dan Abdulazeez (2024) membahas berbagai algoritma Machine Learning untuk klasifikasi penyakit jantung. Kajian tersebut mencakup algoritma Naive Bayes, Decision Tree, Random Forest, Support Vector Machine, dan beberapa metode lainnya.

Liu et al. (2024) melakukan tinjauan sistematis dan meta-analisis terhadap model prediksi risiko penyakit kardiovaskular menggunakan data rekam medis elektronik. Hasil penelitian menunjukkan bahwa Machine Learning memiliki potensi dalam meningkatkan kemampuan prediksi risiko penyakit kardiovaskular, meskipun performanya dapat bervariasi berdasarkan dataset dan metode yang digunakan.

Berdasarkan penelitian terdahulu tersebut, algoritma Machine Learning dapat dimanfaatkan untuk membantu proses klasifikasi penyakit jantung. Oleh karena itu, praktikum ini menerapkan Naive Bayes dan Decision Tree untuk membandingkan performa kedua algoritma pada dataset Heart Disease.

BAB III

HASIL

3.1 Lingkungan dan Persiapan Praktikum

Praktikum Ujian Tengah Semester Pembelajaran Mesin dilaksanakan menggunakan bahasa pemrograman Python melalui aplikasi Jupyter Notebook. Dalam praktikum ini digunakan beberapa library, yaitu Pandas dan NumPy untuk pengolahan data, Matplotlib dan Seaborn untuk visualisasi, serta Scikit-learn untuk proses preprocessing, pelatihan model, dan evaluasi hasil klasifikasi.

Dataset yang digunakan adalah dataset Heart Disease dengan nama file 01_heart_disease.csv. Dataset tersebut memiliki 918 baris data dan 12 kolom yang terdiri dari 11 atribut fitur dan satu atribut target, yaitu HeartDisease.

Dua algoritma yang digunakan dalam praktikum ini adalah Gaussian Naive Bayes dan Decision Tree Classifier.

3.2 Import Library dan Membaca Dataset

Tahap pertama adalah mengimpor library yang dibutuhkan selama proses praktikum. Library Pandas digunakan untuk membaca dan mengolah dataset, NumPy digunakan untuk operasi numerik, sedangkan Matplotlib dan Seaborn digunakan untuk membuat visualisasi data.

Selanjutnya, dataset Heart Disease dibaca menggunakan fungsi read_csv() sehingga data dapat ditampilkan dan dianalisis menggunakan Python.

Hasil yang diperoleh:

Dataset berhasil dibaca dengan jumlah 918 baris dan 12 kolom. Data tersebut berisi informasi mengenai usia, jenis kelamin, jenis nyeri dada, tekanan darah, kolesterol, dan atribut lainnya yang berkaitan dengan kondisi jantung.

```
# NIM: 202431054

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

print("Library berhasil diimport!")
```

```
Library berhasil diimport!
```

```
# NIM: 202431054

df = pd.read_csv("01_heart_disease.csv")

print("Dataset berhasil dibaca!")
```

```
Dataset berhasil dibaca!
```

3.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis dilakukan untuk memahami karakteristik dataset sebelum proses preprocessing dan pelatihan model.

3.3.1 Melihat Sampel Dataset

Perintah head() digunakan untuk menampilkan lima baris pertama dari dataset. Hasilnya memperlihatkan contoh informasi pasien beserta atribut yang terdapat di dalam dataset.

```
# NIM: 202431054

df.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

3.3.2 Memeriksa Ukuran dan Nama Kolom

Pemeriksaan ukuran dataset dilakukan menggunakan atribut shape, sedangkan nama atribut diperiksa menggunakan columns.

Hasil pemeriksaan menunjukkan bahwa dataset memiliki 918 baris dan 12 kolom. Kolom tersebut terdiri atas Age, Sex, ChestPainType, RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope, dan HeartDisease.

```
# NIM: 202431054

print("Jumlah Baris dan Kolom:")
print(df.shape)
```

```
Jumlah Baris dan Kolom:
(918, 12)
```

```
# NIM: 202431054

print("Daftar Kolom Dataset:")
print(df.columns.tolist())
```

```
Daftar Kolom Dataset:
['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS', 'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope', 'HeartDisease']
```

3.3.3 Memeriksa Tipe Data

Pemeriksaan tipe data dilakukan untuk mengetahui apakah setiap atribut termasuk numerik atau kategorikal. Hasilnya menunjukkan bahwa dataset memiliki atribut numerik seperti Age, RestingBP, Cholesterol, MaxHR, dan Oldpeak, serta atribut kategorikal seperti Sex, ChestPainType, RestingECG, ExerciseAngina, dan ST_Slope.

Informasi tipe data ini digunakan sebagai dasar dalam menentukan metode preprocessing yang sesuai.

```
# NIM: 202431054

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    str
 2   ChestPainType   918 non-null    str
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    str
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    str
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    str
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 97.6 KB
```

3.3.4 Memeriksa Missing Value dan Data Duplikat

Pemeriksaan missing value dilakukan untuk mengetahui jumlah nilai kosong pada setiap kolom. Selain itu, pemeriksaan duplikasi dilakukan untuk mengetahui apakah terdapat baris data yang sama.

Berdasarkan hasil pemeriksaan, seluruh kolom memiliki 0 missing value dan tidak ditemukan data duplikat. Dengan demikian, dataset tidak memiliki nilai kosong yang terdeteksi secara langsung maupun baris duplikat.

```
# NIM: 202431054

print("Jumlah Missing Value:")
print(df.isnull().sum())
```

```
Jumlah Missing Value:
Age               0
Sex               0
ChestPainType     0
RestingBP         0
Cholesterol       0
FastingBS         0
RestingECG        0
MaxHR             0
ExerciseAngina    0
Oldpeak           0
ST_Slope          0
HeartDisease      0
dtype: int64
```

```
# NIM: 202431054

print("Jumlah Data Duplikat:")
print(df.duplicated().sum())
```

```
Jumlah Data Duplikat:
0
```

3.3.5 Statistik Deskriptif

Statistik deskriptif digunakan untuk mengetahui gambaran umum data numerik, seperti nilai rata-rata, standar deviasi, nilai minimum, dan maksimum.

Informasi tersebut membantu memahami rentang nilai setiap atribut serta karakteristik data yang digunakan dalam praktikum.

```
# NIM: 202431054

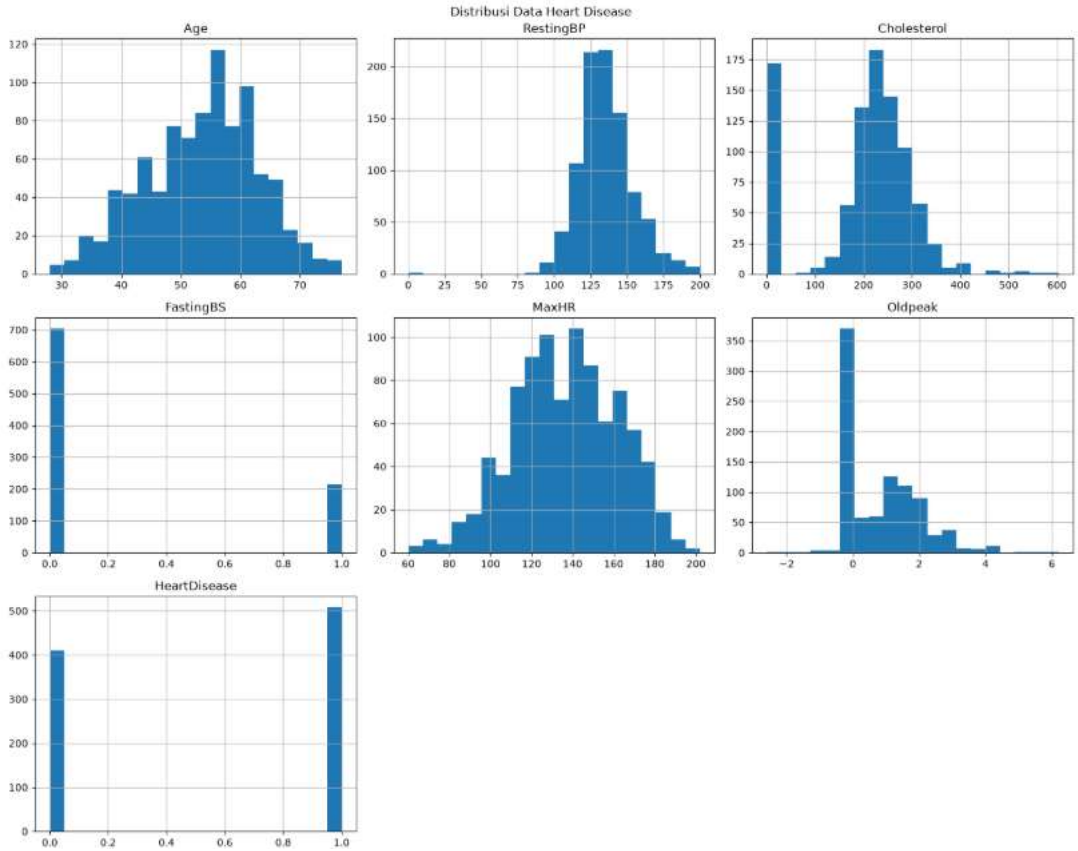
df.describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

3.3.6 Visualisasi Distribusi Data

Visualisasi distribusi dilakukan menggunakan histogram untuk melihat persebaran nilai pada atribut numerik. Visualisasi ini membantu mengetahui pola distribusi data dan perbedaan karakteristik antaratribut.

Selain itu, heatmap korelasi digunakan untuk melihat hubungan antarvariabel numerik berdasarkan nilai korelasinya.





3.3.7 Distribusi Target HeartDisease

Kolom HeartDisease merupakan variabel target yang digunakan dalam proses klasifikasi. Nilai 0 menunjukkan kategori tidak terdapat penyakit jantung berdasarkan label dataset, sedangkan nilai 1 menunjukkan kategori terdapat penyakit jantung berdasarkan label dataset.

Visualisasi distribusi target dilakukan untuk mengetahui persebaran jumlah data pada kedua kelas tersebut.

```
# NIM: 202431054

print("DISTRIBUSI TARGET HEART DISEASE")
print(df["HeartDisease"].value_counts())

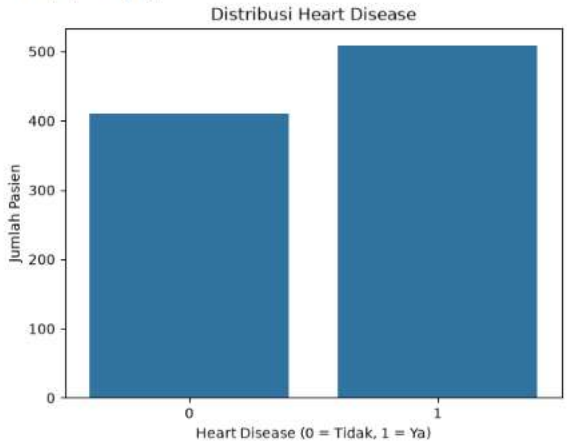
print("\nPERSENTASE TARGET (%)")
print(df["HeartDisease"].value_counts(normalize=True) * 100)

sns.countplot(data=df, x="HeartDisease")

plt.title("Distribusi Heart Disease")
plt.xlabel("Heart Disease (0 = Tidak, 1 = Ya)")
plt.ylabel("Jumlah Pasien")
plt.show()
```

```
DISTRIBUSI TARGET HEART DISEASE
HeartDisease
1    508
0    410
Name: count, dtype: int64

PERSENTASE TARGET (%)
HeartDisease
1    55.337691
0    44.662309
Name: proportion, dtype: float64
```



3.3.8 Pemeriksaan Nilai Tidak Valid dan Outlier

Pemeriksaan nilai dilakukan untuk mengidentifikasi kemungkinan nilai yang tidak valid pada atribut tertentu, seperti RestingBP dan Cholesterol. Pemeriksaan outlier dilakukan menggunakan boxplot pada beberapa atribut numerik.

Hasil visualisasi boxplot dapat membantu mengidentifikasi nilai yang jauh dari persebaran umum data. Nilai yang terdeteksi sebagai outlier tidak secara otomatis dihapus karena perlu mempertimbangkan karakteristik dan konteks data.

```
# NIM: 202431054

kolom_pemeriksaan = ["RestingBP", "Cholesterol", "MaxHR"]

print("PEMERIKSAAN NILAI NOL")

for kolom in kolom_pemeriksaan:
    print(kolom, ":", (df[kolom] == 0).sum())
```

```
PEMERIKSAAN NILAI NOL
RestingBP : 1
Cholesterol : 172
MaxHR : 0
```

```python
# NIM: 202431054

kolom_kategori = [
    "Sex",
    "ChestPainType",
    "RestingECG",
    "ExerciseAngina",
    "ST_Slope"
]

for kolom in kolom_kategori:
    print("\nKolom:", kolom)
    print(df[kolom].value_counts())
```

```
Kolom: Sex
Sex
M    725
F    193
Name: count, dtype: int64

Kolom: ChestPainType
ChestPainType
ASY    496
NAP    203
ATA    173
TA      46
Name: count, dtype: int64

Kolom: RestingECG
RestingECG
Normal    552
LVH       188
ST        178
Name: count, dtype: int64

Kolom: ExerciseAngina
ExerciseAngina
N    547
Y    371
Name: count, dtype: int64

Kolom: ST_Slope
ST_Slope
Flat    460
Up      395
Down     63
Name: count, dtype: int64
```

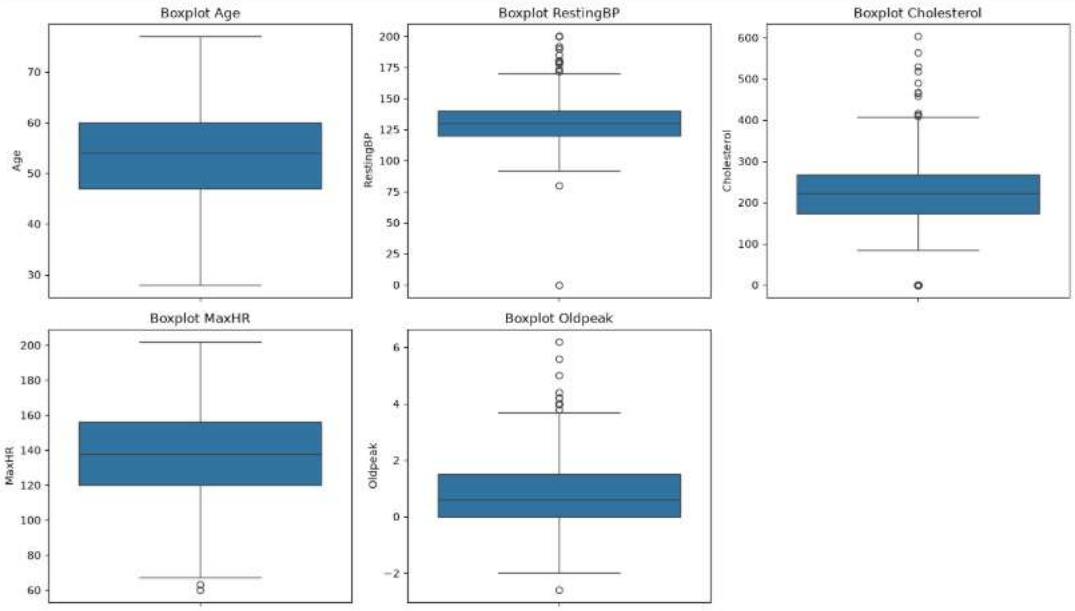
```python
# NIM: 202431054

kolom_numerik = [
    "Age",
    "RestingBP",
    "Cholesterol",
    "MaxHR",
    "Oldpeak"
]

plt.figure(figsize=(14, 8))

for i, kolom in enumerate(kolom_numerik, 1):
    plt.subplot(2, 3, i)
    sns.boxplot(y=df[kolom])
    plt.title("Boxplot " + kolom)

plt.tight_layout()
plt.show()
```



3.4 Data Preprocessing

Preprocessing dilakukan untuk mempersiapkan dataset sebelum digunakan dalam proses pelatihan model. Tahapan ini mencakup pemisahan fitur dan target, pembagian data latih dan data uji, penanganan nilai tidak valid, serta encoding atribut kategorikal.

3.4.1 Pemisahan Fitur dan Target

Pada tahap ini, kolom HeartDisease digunakan sebagai target atau variabel yang akan diprediksi. Sementara itu, seluruh kolom selain HeartDisease digunakan sebagai fitur atau variabel input.

Pemisahan dilakukan agar model dapat mempelajari hubungan antara karakteristik pasien dengan kelas target.

3.4.2 Pembagian Data Latih dan Data Uji

Dataset dibagi menjadi 80% data latih dan 20% data uji. Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk mengukur performa model terhadap data yang belum digunakan selama pelatihan.

Pembagian data dilakukan menggunakan train_test_split dengan parameter random_state=42 dan stratify=y. Parameter stratifikasi digunakan untuk menjaga proporsi kelas target pada data latih dan data uji.

3.4.3 Penanganan Data dan Encoding

Penanganan data dilakukan menggunakan pipeline preprocessing. Pada atribut numerik, digunakan median imputation untuk menangani nilai yang ditandai sebagai missing value. Pada atribut kategorikal, digunakan most frequent imputation dan One-Hot Encoding.

Proses preprocessing dipelajari dari data latih dan kemudian diterapkan pada data uji. Hal ini bertujuan menghindari kebocoran informasi atau data leakage selama proses pelatihan model.

3.5 Implementasi Algoritma Naive Bayes

Algoritma pertama yang diterapkan adalah Gaussian Naive Bayes. Algoritma ini menggunakan pendekatan probabilitas berdasarkan Teorema Bayes untuk menentukan kelas yang paling mungkin berdasarkan fitur yang tersedia.

Model dilatih menggunakan data latih yang telah melalui preprocessing. Setelah proses pelatihan selesai, model digunakan untuk memprediksi kelas pada data uji.

Berdasarkan hasil pengujian, algoritma Naive Bayes memperoleh akurasi sebesar **86,96%**.

Hasil classification report menunjukkan bahwa:

- Kelas 0 memperoleh precision 0,85, recall 0,87, dan F1-score 0,86.
- Kelas 1 memperoleh precision 0,89, recall 0,87, dan F1-score 0,88.
- Accuracy keseluruhan sebesar 0,87 atau sekitar 86,96%.

Hasil tersebut menunjukkan bahwa model Naive Bayes mampu melakukan klasifikasi dengan performa yang cukup baik pada data pengujian.

```python
# NIM: 202431054

from sklearn.naive_bayes import GaussianNB

# Membuat pipeline Naive Bayes
model_nb = Pipeline([
    ("preprocessing", preprocessor),
    ("model", GaussianNB())
])

# Melatih model
model_nb.fit(X_train, y_train)

print("Training Naive Bayes berhasil!")
```

```
Training Naive Bayes berhasil!
```

```python
# NIM: 202431054

y_pred_nb = model_nb.predict(X_test)

print("Hasil prediksi Naive Bayes:")
print(y_pred_nb)
```

```
Hasil prediksi Naive Bayes:
[1 0 1 1 0 1 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 1 1 1
 1 0 1 1 0 1 0 0 1 1 0 1 0 1 0 0 1 1 0 1 0 1 0 0 1 0 1 0 1 1 1 0 0 0 1 0 1
 0 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 1 0 0 1 1 1 1 0 1 1 1 1 0 1 1 1 0 0 1 1
 0 0 1 1 0 1 0 1 1 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 0 0 0 0 1 1 0 1 1 1 1 0 0
 1 1 1 0 1 0 1 1 0 0 1 1 1 1 0 0 1 0 1 1 0 1 1 0 1 1 1 1 0 0 0 0 1 1 1 1]
```

```python
# NIM: 202431054

from sklearn.metrics import accuracy_score

akurasi_nb = accuracy_score(y_test, y_pred_nb)

print("AKURASI NAIVE BAYES")
print("==================")
print("Akurasi:", round(akurasi_nb * 100, 2), "%")
```

```
AKURASI NAIVE BAYES
==================
Akurasi: 86.96 %
```

```python
# NIM: 202431054

from sklearn.metrics import classification_report

print("CLASSIFICATION REPORT NAIVE BAYES")
print(classification_report(y_test, y_pred_nb))
```

```
CLASSIFICATION REPORT NAIVE BAYES
              precision    recall  f1-score   support

           0       0.85      0.87      0.86        82
           1       0.89      0.87      0.88       102

    accuracy                           0.87       184
   macro avg       0.87      0.87      0.87       184
weighted avg       0.87      0.87      0.87       184
```

```python
# NIM: 202431054

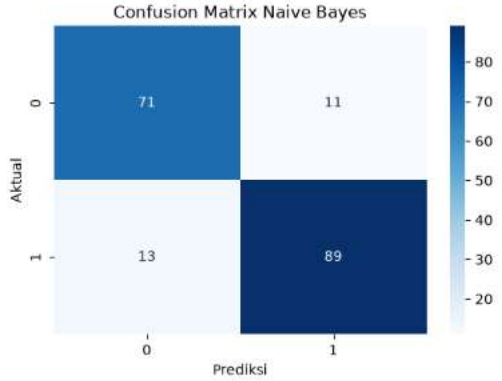
from sklearn.metrics import confusion_matrix

cm_nb = confusion_matrix(y_test, y_pred_nb)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_nb,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Confusion Matrix Naive Bayes")
plt.xlabel("Prediksi")
plt.ylabel("Aktual")
plt.show()
```



3.6 Implementasi Algoritma Decision Tree

Algoritma kedua yang diterapkan adalah Decision Tree Classifier. Algoritma ini menggunakan struktur pohon keputusan untuk menentukan kelas berdasarkan pembagian atribut tertentu.

Pada praktikum ini, Decision Tree menggunakan kriteria Gini untuk menentukan pemisahan data. Model dilatih menggunakan data latih yang telah melalui preprocessing, kemudian digunakan untuk melakukan prediksi terhadap data uji.

Berdasarkan hasil pengujian, algoritma Decision Tree memperoleh akurasi sebesar **80,43%**.

Hasil classification report menunjukkan bahwa:

- Kelas 0 memperoleh precision 0,78, recall 0,78, dan F1-score 0,78.
- Kelas 1 memperoleh precision 0,82, recall 0,82, dan F1-score 0,82.
- Accuracy keseluruhan sebesar 0,80 atau sekitar 80,43%.

Hasil tersebut menunjukkan bahwa Decision Tree mampu melakukan klasifikasi terhadap dataset Heart Disease, meskipun akurasinya lebih rendah dibandingkan Naive Bayes pada eksperimen ini.

```python
# NIM: 202431054

from sklearn.tree import DecisionTreeClassifier

# Membuat pipeline Decision Tree
model_dt = Pipeline([
    ("preprocessing", preprocessor),
    ("model", DecisionTreeClassifier(
        criterion="gini",
        random_state=42
    ))
])

# Melatih model
model_dt.fit(X_train, y_train)

print("Training Decision Tree berhasil!")
```

```
Training Decision Tree berhasil!
```

```python
# NIM: 202431054

y_pred_dt = model_dt.predict(X_test)

print("Hasil prediksi Decision Tree:")
print(y_pred_dt)
```

```
Hasil prediksi Decision Tree:
[1 1 1 0 1 1 0 1 0 1 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 1 0 1 1 0 0 1 0 0 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 0 1 0 0 0 1 1 1 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 1 1 1
 0 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 1
 1 0 1 1 1 1 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0 0 1 1 0 0 1 1 1 1 0 1 1 1 1 0
 0 0 1 0 1 0 1 1 0 0 1 1 0 1 0 1 1 1 0 0 1 0 1 1 1 1 1 1 0 1 1 0 0 0 0 1 1 1 1]
```

```python
# NIM: 202431054

akurasi_dt = accuracy_score(y_test, y_pred_dt)

print("AKURASI DECISION TREE")
print("====================")
print("Akurasi:", round(akurasi_dt * 100, 2), "%")
```

```
AKURASI DECISION TREE
====================
Akurasi: 80.43 %
```

```python
# NIM: 202431054

print("CLASSIFICATION REPORT DECISION TREE")

print(classification_report(y_test, y_pred_dt))
```

```
CLASSIFICATION REPORT DECISION TREE
              precision    recall  f1-score   support

           0       0.78      0.78      0.78        82
           1       0.82      0.82      0.82       102

    accuracy                           0.80       184
   macro avg       0.80      0.80      0.80       184
weighted avg       0.80      0.80      0.80       184
```

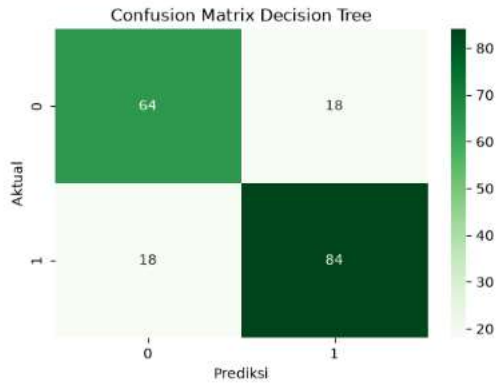
```python
# NIM: 202431054

cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    cmap="Greens"
)

plt.title("Confusion Matrix Decision Tree")
plt.xlabel("Prediksi")
plt.ylabel("Aktual")
plt.show()
```



3.7 Perbandingan Hasil Evaluasi Model

Setelah kedua algoritma selesai dilatih dan diuji, dilakukan perbandingan nilai akurasi untuk mengetahui model yang menghasilkan performa terbaik.

Hasil perbandingan akurasi adalah sebagai berikut:

Algoritma	**Akurasi**
Naive Bayes	86,96%
Decision Tree	80,43%

Berdasarkan hasil tersebut, Naive Bayes memiliki akurasi lebih tinggi sebesar 6,52 poin persentase dibandingkan Decision Tree.

Perbedaan hasil ini menunjukkan bahwa kedua algoritma memiliki performa yang berbeda dalam mempelajari pola pada dataset Heart Disease. Pada pembagian data dan konfigurasi yang digunakan, Naive Bayes memberikan hasil klasifikasi yang lebih baik berdasarkan nilai akurasi dan classification report.

```
# NIM: 202431054

hasil_perbandingan = pd.DataFrame({
    "Algoritma": ["Naive Bayes", "Decision Tree"],
    "Akurasi": [
        akurasi_nb * 100,
        akurasi_dt * 100
    ]
})

print("PERBANDINGAN AKURASI MODEL")
display(hasil_perbandingan)

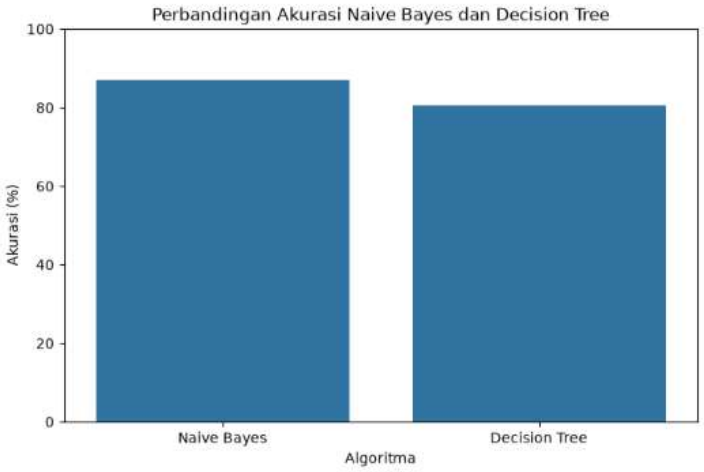
plt.figure(figsize=(8, 5))

sns.barplot(
    data=hasil_perbandingan,
    x="Algoritma",
    y="Akurasi"
)

plt.title("Perbandingan Akurasi Naive Bayes dan Decision Tree")
plt.ylabel("Akurasi (%)")
plt.ylim(0, 100)
plt.show()
```

```
PERBANDINGAN AKURASI MODEL
```

	Algoritma	Akurasi
0	Naive Bayes	86.956522
1	Decision Tree	80.434783




```
# NIM: 202431054

if akurasi_nb > akurasi_dt:
    print("Naive Bayes memiliki akurasi lebih tinggi.")
elif akurasi_dt > akurasi_nb:
    print("Decision Tree memiliki akurasi lebih tinggi.")
else:
    print("Kedua algoritma memiliki akurasi yang sama.")

print("\nAkurasi Naive Bayes:",
      round(akurasi_nb * 100, 2), "%")

print("Akurasi Decision Tree:",
      round(akurasi_dt * 100, 2), "%")
```

```
Naive Bayes memiliki akurasi lebih tinggi.

Akurasi Naive Bayes: 86.96 %
Akurasi Decision Tree: 80.43 %
```

3.8 Analisis Hasil Praktikum

Berdasarkan seluruh tahapan praktikum, dataset Heart Disease berhasil diproses dan digunakan untuk membangun dua model klasifikasi, yaitu Naive Bayes dan Decision Tree.

Hasil pengujian menunjukkan bahwa Naive Bayes memperoleh akurasi sebesar 86,96%, sedangkan Decision Tree memperoleh akurasi sebesar 80,43%. Selain menghasilkan akurasi yang lebih tinggi, Naive Bayes juga memperoleh nilai precision, recall, dan F1-score yang lebih tinggi pada kedua kelas dibandingkan Decision Tree.

Dengan demikian, berdasarkan hasil eksperimen yang dilakukan, Naive Bayes merupakan algoritma dengan performa terbaik untuk klasifikasi dataset Heart Disease pada konfigurasi pengujian ini.

Meskipun demikian, hasil klasifikasi tersebut merupakan hasil eksperimen pembelajaran mesin dan tidak dapat digunakan sebagai pengganti diagnosis medis oleh tenaga kesehatan.

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa proses klasifikasi menggunakan pembelajaran mesin dapat diterapkan pada dataset Heart Disease. Tahapan praktikum meliputi eksplorasi data, preprocessing, pembagian data latih dan data uji, pelatihan model, serta evaluasi hasil klasifikasi.

Pada eksperimen ini digunakan dua algoritma, yaitu Naive Bayes dan Decision Tree. Berdasarkan hasil pengujian terhadap 184 data uji, algoritma Naive Bayes memperoleh akurasi sebesar 86,96%, sedangkan algoritma Decision Tree memperoleh akurasi sebesar 80,43%.

Dengan demikian, Naive Bayes menghasilkan akurasi lebih tinggi sebesar 6,52 poin persentase dibandingkan Decision Tree. Berdasarkan hasil tersebut, Naive Bayes menjadi algoritma dengan performa terbaik pada eksperimen yang dilakukan.

Praktikum ini memberikan pemahaman mengenai tahapan penerapan pembelajaran mesin serta cara membandingkan performa algoritma klasifikasi berdasarkan hasil evaluasi model.

DAFTAR PUSTAKA

1. Mao, Y., Jimma, B. L., & Mihretie, T. B. (2025). Machine learning algorithms for heart disease diagnosis: A systematic review. Current Problems in Cardiology, 50(8), 103082. https://doi.org/10.1016/j.cpcardiol.2025.103082.

2. Duhok, M. H., & Abdulazeez, A. M. (2024). A review of heart disease classification base on machine learning algorithms. The Indonesian Journal of Computer Science, 13(2). https://doi.org/10.33022/ijcs.v13i2.3923.

3. Liu, T., Krentz, A., Lu, L., & Curcin, V. (2024). Machine learning based prediction models for cardiovascular disease risk using electronic health records data: Systematic review and meta-analysis. European Heart Journal – Digital Health, 6(1), 7–22. https://doi.org/10.1093/ehjdh/ztae080.

4. Banerjee, T., & Paçal, İ. (2025). A systematic review of machine learning in heart disease prediction. Turkish Journal of Biology, 49(5), 600–634. https://doi.org/10.55730/1300-0152.2766